Smart parking occupancy and gate

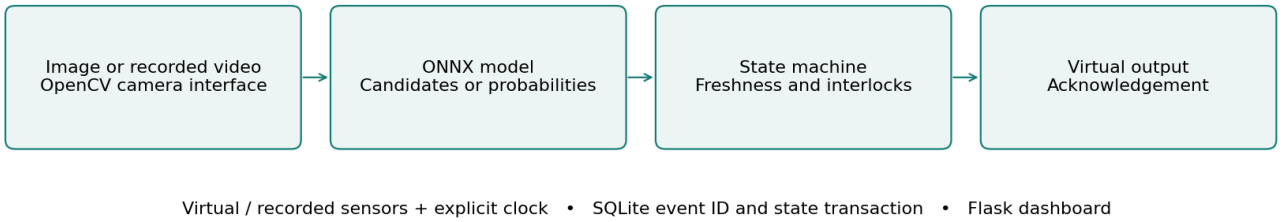
Technical report and reproducible evaluation

11 November 2026 | Abhijith Viswanathan | AI-assisted implementation

A camera and four virtual proximity readings estimate parking-space availability. Only agreement produces a free or occupied decision; disagreement reserves the space. A separate gate state machine requires fresh clear obstruction evidence before closing, including during manual requests. This is a software simulation prototype, not a certified gate controller.

Raspberry Pi targeted, desktop simulation tested. No Pi, real sensor, electrical power, camera timing, motor or physical safety validation was performed. This is an AI-assisted personal engineering project; it does not establish professional seniority or independent authorship.

Architecture



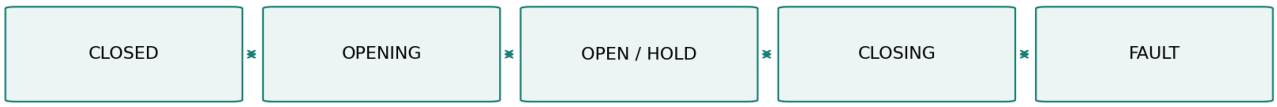
The application keeps perception, control and hardware interfaces separate. Virtual inputs and the explicit clock make decisions replayable; SQLite commits event identifiers and controller state together. The dashboard runs locally and never opens a webcam unless explicitly enabled.

Main measured comparison

Vehicle detection F1: 0.386 to 0.581 score. 78 COCO test images; IoU >= 0.5; threshold 0.25; 320 to 640 pixels.

The following pages separate real image evaluation, host execution time, scripted control outcomes and future physical validation. Raw observations, exact selections, source attribution and regression logs accompany this report.

Control behavior and recovery



State inventory; transition guards and recovery paths are defined in the policy table and core.py

1. Space hysteresis uses distance below 70 cm as near and above 90 cm as far; valid distance range is 2 to 400 cm. Camera and sensor agreement produces the decision.

2. CLOSED transitions to OPENING only when a space is confirmed free; the virtual opening takes two seconds.

3. OPEN or HOLD requires two seconds of fresh clear evidence to enter CLOSING. Any obstruction or invalid evidence during closure moves to HOLD.

4. Actuator failure enters FAULT. Reset requires stable clear evidence and returns to OPEN for inspection. Restart during a gate cycle enters FAULT.

Freshness and persistence

For classroom and parking, real image evidence expires after two simulation seconds at a new check; advancing time cannot make a stored frame fresh. Restart invalidates camera evidence. A gate interrupted during a cycle enters FAULT, and an interrupted waste move requires review. Manual simulation inputs are explicitly labeled fixtures.

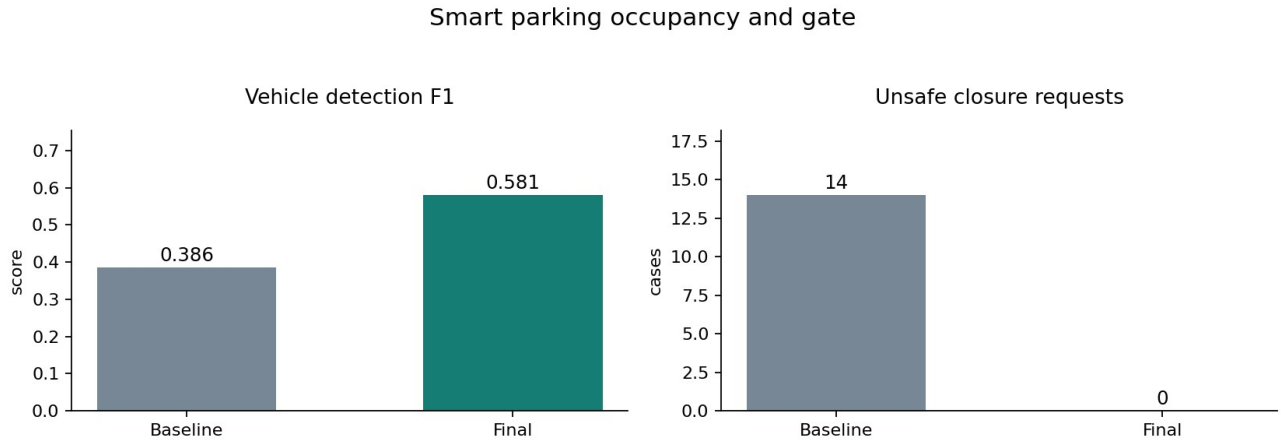
The classroom and parking tracker uses class-aware one-to-one overlap association, a five-second expiry, two-hit confirmation and a line deadband. An 80-frame synthetic trajectory produced one unique ID and one expected entry. This verifies mechanics, not tracking accuracy on crowded video. IDs and entry counters persist, but active tracks do not survive restart.

A state change and its event ID are stored in one SQLite transaction. Duplicate event IDs are ignored. Failed persistence rolls back in-memory controller state and newly accepted inference/tracker state. This cannot undo a real actuator movement; the physical design requires independent acknowledgement and reconciliation.

State-machine source: docs/state-machine.mmd. Controllers: lab/core.py. Hardware interfaces: lab/hal.py. The current app uses serialized, synchronous inference and virtual-clock control; it is not a real-time motor controller.

Models data and measured tradeoffs

YOLO11n detects car, motorcycle, bus and truck; person detections feed the obstruction flag. The weights remain pretrained. Float32 ONNX at 640 replaces a 320-pixel baseline, with threshold 0.25 selected on 42 validation images. Detection matches use class-aware greedy IoU >= 0.5 on 78 test images. This is not official COCO AP; ignored crowd regions can still generate false positives.



Higher resolution increased vehicle F1 from 0.386 to 0.581 at roughly 3.6 times detector latency. Conservative fusion eliminated 15 false-free slot-seconds on the scripted trace, but only 83.33% of slot-seconds received a definite decision. Accuracy on decided slots was 100%; counting unknowns as errors gives 83.33%, below the baseline 93.75%. Reserving uncertainty costs availability.

The four image regions are example vertical lanes, not calibrated real parking spaces. COCO object F1 is not parking occupancy accuracy. Only one motorcycle ground-truth instance occurs in the held-out test. Reflections, occlusion and shadows were explored with transformations, not a labeled parking deployment dataset. A stop/HOLD software state does not physically reopen a gate. Independent obstruction hardware, limit switches and electrical interlocks remain required.

Measurement protocol and verification

Metric	Baseline	Final	Unit
Vehicle detection F1	0.386	0.581	score
False free availability	15.000	0.000	slot-seconds
Unsafe closure requests	14.000	0.000	cases

Vehicle detection F1: 78 COCO test images; IoU >= 0.5; threshold 0.25; 320 to 640 pixels.

False free availability: 480 synthetic slot-seconds; final decision coverage 83.33%.

Unsafe closure requests: 30 scripted closure cases; not a physical safety rating.

Complete local inference requests: median 52.18 ms and p95 54.55 ms over 30 repeated requests after one cold request (73.81 ms). Flask test-client timing includes image decode, inference, annotation, controller update, SQLite and JSON; it excludes network and browser rendering. The same selected image was repeated. CPU and cumulative process memory samples are in evidence/request-timing.json.

Environment: macOS 26.2 arm64, Python 3.12.14, ONNX Runtime 1.21.0 CPU, OpenCV 4.11.0.86 and NumPy 2.2.6. Detector timing separately compares 36 calls at each size; median 13.31 ms at 320 and 48.50 ms at 640. Short host runs do not predict Pi throughput, thermal behavior or sustained response bounds.

Regression result: 56 shared cases passed, covering state guards, invalid and stale input, manual requests, duplicate IDs, restart, large time steps, failed commits and HTTP input boundaries. The same suite is included in each standalone project; these are not 168 distinct tests. Browser checks exercised real inference and the advertised failure controls.

Version history preserves the initial 38 passing core cases, subsequent API and review regressions, and the final 56-case run. Decoder correction, stale image handling, obstruction-source separation, capacity changing during motion, scenario labeling and restart/rollback fixes are recorded in docs/FAILURES.md. Earlier detection results remain available and are not used as final metrics.

Hardware integration and physical validation

Raspberry Pi 4 or 5, USB camera, four HC-SR04 distance sensors or verified 3.3 V proximity modules, independent break-beam obstruction sensor, small bench servo, two limit switches, separate regulated servo supply and common signal ground.

Example BCM allocation: trigger/echo pairs (5,6), (13,19), (20,21), (23,24); obstruction input GPIO17; servo signal GPIO18; open/closed limit inputs GPIO22/27. Use an appropriate divider or level shifter on every 5 V echo line. GPIO numbering is BCM, not physical header numbering.

Interface	Implemented now	Required before physical use
Camera	Recorded files and explicit USB camera option	Capture cadence and disconnect validation
Sensor readings	Virtual and recorded value valid at records	Calibrated GPIO or serial acquisition
Outputs	VirtualOutput and optional GPIO/servo classes	State-to-pin mapping and physical acknowledgement
Timing	VirtualClock and MonotonicClock interfaces	Separate control scheduling and deadline measurements
Persistence	SQLite event and state transaction	Power-loss and incomplete-actuation reconciliation

Required bench tools are a digital multimeter, logic analyzer and USB power meter; an oscilloscope is useful for supply transients. Use BCM numbering, verify 3.3 V GPIO levels and level-shift 5 V echoes. A servo needs a suitable separate supply and common signal ground. The lighting demonstration uses a low-voltage LED, and the gate is an unloaded bench mechanism.

The shipped app does not activate GPIO. Optional adapter classes are integration building blocks. A target runner must timestamp real acquisitions, disable synthetic acknowledgements, map outputs, and keep interlock checks independent of inference. Direct HX711 support is not included; weight expects a calibrated serial bridge.

Validate in stages: inspect disconnected rails; calibrate sensors; inject wiring and sensor faults; measure LED/servo command and stop timing; run sustained capture under load; interrupt power during writes and movement; then collect deployment data split by complete recording and location. Every physical test remains pending.

Hardware sources: Raspberry Pi computer documentation and gpiozero input/output API documentation. Full URLs, pin notes and the integration checklist are in docs/hardware.md.

Dashboard demonstration and learning

Panel	Interpretation
Input and detections	Uploaded image/recorded frame with boxes, or a clearly labeled synthetic scene.
Observation	Controller state, distinct from detector confidence.
Count and actuator	Candidate count/free slots/acknowledged disposals; virtual output only.
Last inference	Measured model/preprocessing time on this host; not Pi latency.
Latest decision and sensors	Reason, validity and current virtual sensor values.
Observation history	Count versus explicit simulation time; duplicate timestamps may overlap.
Experiment controls	Fault scenarios, manual requests, threshold and applicable interval controls.
Event ledger	Latest 30 rendered events from a 500-event export window; database retains all events.
Recorded benchmarks	Frozen historical results and JSON export; not recomputed by dashboard controls.

Click conflict to reserve a disagreement. Click obstruction to replay opening, closing and obstruction, ending in HOLD. Restore devices, request close and advance time to demonstrate the clear-evidence delay. Select a real COCO sample and run inference; its regions are examples, not calibrated spaces.

Start with Python 3.12, create .venv, install requirements.txt and run app.py. Control fixtures need no model files. For AI, install requirements-training.txt and run tools/setup_models.py and tools/fetch_samples.py. The trained waste head is included; source packages omit pretrained backbones and COCO photos. Detailed commands and troubleshooting are in docs/setup-and-demo.md.

Use docs/mentor-guide.md for the code walkthrough, engineering alternatives, ten explained interview questions, modifications and three measured resume bullets. Begin with core.py and its tests, then trace a dashboard action through runtime.py, inference.py and the persistence interface.

Attribution: Ultralytics YOLO11, COCO, TrashNet by Gary Thung and Mindy Yang, and torchvision MobileNetV3. License texts and source links are retained in NOTICE.md, licenses and docs/model-and-data.md. No identity recognition is implemented. The project demonstrates reproducible simulation work; physical deployment and senior professional experience are not claimed.